DBMS

Assignment 1

Name: Sidhartha Durgam **Roll No.:** DA24B003

Deliverable A – Assumptions and Scope

A.1 – Assumptions

#	Where in brief	Ambiguity, as a question	My assumption	Why	What breaks if opposite were true
1	§2.1 – Users	Are handles case-sensitive for uniqueness?	Handles are case-insensitive; `Sidhartha` and `sidharthaumner umner` are the same handle.	The brief cites support tickets where handles differ only in capitalisation.	Case-sensitive uniqueness allows visually identical duplicates to coexist.
2	§2.1 – Users	Can a handle be reused immediately after its account becomes inactive?	A handle remains reserved while its account is deactivated or pending deletion.	Releasing it during the recovery window would allow identity confusion across lifecycle transitions.	The same handle could refer to different accounts at different times, requiring handle-history and time-aware resolution.
3	§2.1 – Users	When an ML-inferred interest is refreshed weekly, is the previous inference replaced or retained?	Each inference is retained historically with its confidence score and week period.	The ML team must reconstruct a user's state at any past timestamp; overwriting destroys that information.	Only storing the latest inference causes temporal leakage: a later confidence score would be used for an earlier event.
4	§2.2 – Creators and Content	Does a monetisation-tier record represent the current tier, a change event, or a continuous period?	Each record represents one continuous validity period for that tier.	Finance must know which tier a creator held at a past time; validity periods make this a direct bounded query.	Current-only storage loses historical tiers; event-only storage requires reconstructing intervals from successive events for every lookup.
5	§2.2 – Creators and Content	Are hashtags first-class items separate from the caption text?	Hashtags are extracted and stored separately; the original caption is also retained.	Hashtags currently live inside captions with inconsistent case; separating them enables reliable queries and normalisation.	Caption-only storage requires text parsing and case normalisation for every hashtag query, making consistent matching unreliable.

#	Where in brief	Ambiguity, as a question	My assumption	Why	What breaks if opposite were true
6	§2.2 – Creators and Content	Are moderation decisions overwritten, or is the full history retained?	Every decision is retained as an event; current state is derived from the latest row.	Each decision has an actor and timestamp; history is required for auditability, past-state reconstruction, and the dual-write fix in §2.7.	Overwriting loses the video's moderation state at any past timestamp, making historical analysis, auditing, and the G.3 atomicity demonstration impossible.
7	§2.3 – The Social Graph	When a block breaks a follow, is the follow row deleted or retained with an end time?	The follow row is retained with an `ended_at` timestamp.	A deleted follow is indistinguishable from one that never existed, destroying historical follow-count information.	Hard-deletion means no query can reconstruct that a follow existed before the block; historical follow activity undercounts.
8	§2.4 – Watch Telemetry	Is an impression recorded only when it becomes a view, or always?	Every impression is recorded independently, even those that never reach the 300 ms view threshold.	The brief requires funnel reports to be correct for a creator shown 900 times but watched twice; impression and view must be separate events.	Recording only viewed impressions makes the view/impression ratio uncomputable and drops zero-watch creators from funnel reports.
9	§2.4 – Watch Telemetry	Is the feed position fixed at impression time or mutable as the feed changes?	Feed position is recorded at impression time and never updated.	An impression is a specific exposure event; its position must reflect the recommendation context at that exact moment.	A mutable position would retroactively change historical recommendation analysis and lose the original ranking evidence.
10	§2.4 – Watch Telemetry	When a like is retracted, is the like row deleted or retained alongside a separate retraction record?	The like is retained; the retraction is recorded as a separate negative signal.	The brief states a retraction is "a negative signal in its own right, not merely the absence of a like"; the §2.7 dual-write failure also requires them to be separate events.	Hard-deleting the like and storing only a flag prevents past-state reconstruction and makes the G.3 atomicity demonstration unrepresentable.
11	§2.5 – The Agent Layer	Is the token price applicable to a turn frozen at turn time or computed from the current price?	Each turn stores a reference to the pricing row in effect when the turn occurred.	Finance requires historical costs to be immutable; tying the price to the turn's effective time prevents retroactive changes.	Without freezing the price, any vendor repricing silently alters previously reported turn, session, and monthly costs.

#	Where in brief	Ambiguity, as a question	My assumption	Why	What breaks if opposite were true
12	§2.5 – The Agent Layer	Is the parent-child relationship between nested tool calls explicitly recorded?	Each tool call has at most one parent; a parent may have many children, allowing arbitrary-depth nesting.	The brief requires nesting to arbitrary depth; F11 reconstructs the full tree with a recursive CTE, which needs a parent reference on each child.	Without the parent reference, nesting depth and execution hierarchy cannot be reconstructed, breaking F11.

A.2 – Scope Decisions

1. **Temporal constraint enforcement:** Gap and overlap checking in validity intervals is deferred to application code or bonus triggers. Re-entry: add as a trigger without schema change.

2. **Tool-call nesting depth validation:** No constraint on maximum nesting depth; arbitrary depth is stored as the brief requires. Re-entry: application-layer validation, no schema change needed.

3. **Prompt-template lifecycle management:** The exact text used at each turn is stored (brief §2.5), but creation/archival workflows are not modelled. Re-entry: add a template registry without changing the turn-template link.

Deliverable B – Conceptual Design

B.1 – ER Diagram

Legend

Kind	Meaning
Strong	Independently identifiable; has its own primary key
Weak	Identified only in the context of a parent entity
Event	Records a discrete, timestamped occurrence; never overwritten
Temporal	Stores state over a time period; queried with validity intervals
Associative	Junction entity resolving a many-to-many relationship
Lookup	Small, stable reference table; changes are rare and non-temporal

Crow's-foot notation is used in the diagram: | = exactly one, ∘ = zero, < = many.

Temporal entities are shown with a double border in the diagram.

Weak entities are shown with a dashed border in the diagram.

Entity Inventory

Entity	One row = ...	Identifier	Kind
§2.1 – Users			
AppUser	One registered account. Authentication identity (phone or Google) stored as two attributes; Assumption 1 guarantees one identity per user.	`user_id`	Strong
InterestCategory	One named interest topic available for declaration or inference.	`category_id`	Lookup
DeclaredInterest	One user's explicit declaration of interest in one category. A user declares a category at most once; the brief does not describe undeclaring and re-declaring, so no historical lifecycle is modelled here.	(`user_id`, `category_id`)	Associative
InferredInterest	One weekly ML inference assigning a confidence score for one interest category to one user during one specific week.	`inference_id`	Event
AccountStatusPeriod	One continuous period during which one user held active creator status.	`status_period_id`	Temporal
§2.2 – Creators and Content			
CreatorPeriod	One continuous period during which one user held active creator status. This is kept separate from CreatorTierPeriod because a user may become a creator before being assigned to any monetisation tier; collapsing the two would leave that initial window unrepresentable.	`creator_period_id`	Temporal
CreatorTierPeriod	One continuous period during which one creator occupied one monetisation tier.	`tier_period_id`	Temporal
Video	One uploaded clip with its metadata (owner, duration, caption, audio track reference, upload time).	`video_id`	Strong
AudioTrack	One audio recording, either original to a video or licensed from a catalogue.	`track_id`	Strong
Hashtag	One normalized hashtag string extracted from video captions.	`hashtag_id`	Lookup
VideoHashtag	One video's association with one hashtag.	(`video_id`, `hashtag_id`)	Associative

Entity	One row = ...	Identifier	Kind
ModerationDecision	One moderation event applied to one video, attributable to an automated classifier or a human reviewer, recording the resulting new moderation state.	`decision_id`	Event
§2.3 – The Social Graph			
Follow	One directional follow relationship from one user to another, with a recorded start time and an optional end time set when the relationship is broken by a block.	`follow_id`	Temporal
Block	One block action from one user against another, recorded as a discrete event.	`block_id`	Event
Mute	One mute relationship from one user against another, with a recorded start time and an optional end time.	`mute_id`	Temporal
§2.4 – Watch Telemetry			
Impression	One entry of one video into one user's viewport at one timestamp, capturing the feed position at that moment and the ranking model version.	`impression_id`	Event
ViewSegment	One continuous stretch of playback within one impression, from a playback start offset to a playback end offset.	(`impression_id`, `segment_start_ms`)	Weak
Like	One like signal from one user on one video, retained historically even after retraction.	`like_id`	Event
LikeRetraction	One retraction of one like, recorded as a distinct negative signal in its own right.	`retraction_id`	Event
ExplicitSignal	One non-like, non-share explicit signal (save, comment, follow-from-feed, not-interested, or report) from one user on one video.	`signal_id`	Event
Share	One share action from one user of one video to one named destination (WhatsApp, Instagram, or copied link).	`share_id`	Event
§2.5 – The Agent Layer			
AgentSession	One conversational session between one user and the agent.	`session_id`	Strong

Entity	One row = ...	Identifier	Kind
Turn	One exchange within one session, comprising one user message and one assistant message, linked to the exact prompt template version and the model used.	`turn_id`	Strong
PromptTemplate	One versioned snapshot of a prompt template's exact text.	`template_id`	Event
ModelPricing	One validity period during which one specific model's token rates (input, output, and cached-input) were in effect.	`pricing_id`	Temporal
ToolCall	One tool invocation issued by the assistant during one turn, recording arguments, result, latency, error status, and an optional reference to its parent tool call.	`call_id`	Event
Recommendation	One video recommended by the agent at one shelf position within one turn, recorded independently of whether the user subsequently watches it.	`recommendation_id`	Event
TurnQualityScore	One LLM judge evaluation of one turn, scoring helpfulness, groundedness, and safety.	`score_id`	Event
UserTurnRating	One thumbs-up or thumbs-down given by one user on one turn at one timestamp.	`rating_id`	Event

Relationship Inventory

Relationship	From → To	Cardinality	Participation	Attributes on relationship
declares	AppUser → InterestCategory	0..N : 0..N	both optional	resolved via DeclaredInterest
inferred for	InterestCategory → AppUser	0..N : 0..N	both optional	resolved via InferredInterest; `confidence`, `week_start`
received decision	Video → ModerationDecision	1 : 1..N	Video mandatory; Decision mandatory	`decided_at`, `new_state`, `actor_type`, `actor_id`
follows	AppUser → AppUser	0..N : 0..N	both optional	resolved via Follow; `started_at`, `ended_at`

Relationship	From → To	Cardinality	Participation	Attributes on relationship
blocks	AppUser → AppUser	0..N : 0..N	both optional	resolved via Block; `blocked_at`
mutes	AppUser → AppUser	0..N : 0..N	both optional	resolved via Mute; `started_at`, `ended_at`
yields	Impression → ViewSegment	1 : 0..N	Impression optional; Segment mandatory	`start_ms`, `end_ms`. One impression can yield multiple segments (user returns, clip loops).
retracted by	Like → LikeRetraction	1 : 0..1	Like optional; Retraction mandatory	`retracted_at`. Retraction is a distinct negative event, not a deletion.
parent of	ToolCall → ToolCall	0..1 : 0..N	both optional	self-reference via `parent_call_id`; supports arbitrary-depth nesting (Assumption 12).
led to	Recommendation → Impression	0..1 : 0..N	both optional	The critical link; FK is a nullable column on Impression.
judged by	Turn → TurnQualityScore	1 : 0..1	both optional	`helpfulness`, `groundedness`, `safety`, `scored_at`
uses audio	Video → AudioTrack	0..1 : 0..N	Video optional; Track optional	—
priced against	Turn → ModelPricing	N : 1	Turn mandatory; Pricing mandatory	`pricing_id` frozen at insert time; rates never recomputed live (Assumption 11).

B.2 – Temporal Register

For each entity, exactly one of three strategies is chosen. **O** = overwrite in place (mutable current state only), **V** = validity intervals (state over a time period), **A** = append-only event log (immutable historical records).

Entity	O	V	A	Justification
AppUser	✓			Mutable display fields are current-value only; no consumer requires their history.
InterestCategory	✓			Stable lookup; past InferredInterest rows reference by surrogate key, so in-place correction is safe.

Entity	O	V	A	Justification
DeclaredInterest	✓			A user declares a category at most once (no undeclare-and-redeclare lifecycle is modelled). The association is either present or absent; no history of changes is required.
InferredInterest			✓	Each weekly ML run appends a new row with its own confidence score and week period. Past rows are never modified. Overwriting would destroy the exact confidence value knowable at a past timestamp, causing temporal leakage in ML training data (Assumption 3).
AccountStatusPeriod		✓		An account moves between three states over its lifecycle. The 30-day deletion window requires knowing exactly when the account entered pending_deletion. Validity intervals support a direct point-in-time lookup without scanning events.
CreatorPeriod		✓		A user may become and later cease to be a creator. Validity intervals allow the ML team to determine whether a user was a creator at a past timestamp without reconstructing event sequences. Kept separate from CreatorTierPeriod because creator status may exist before any tier is assigned.
CreatorTierPeriod		✓		Finance must answer "which tier was this creator in at a past time?" Validity intervals make this a direct bounded query on `valid_from`/`valid_to` (Assumption 4; brief §2.2).
Video	✓			Video metadata (duration, caption, owner, audio track, upload time) is written once and the brief gives no requirement to version or retain its history. The current moderation state is not stored here; it is derived from the latest ModerationDecision row, removing the dual-write failure entirely.
AudioTrack	✓			Track metadata is stable once ingested; no temporal requirement is stated.
Hashtag	✓			Normalized hashtag text is a lookup value; no temporal requirement is stated.
VideoHashtag	✓			The association between a video and a hashtag is written once at extraction time and does not change.
ModerationDecision			✓	Every decision is an immutable event; current state is the `new_state` of the latest row (Assumption 6), eliminating the dual-write failure from §2.7.
Follow		✓		A follow has a start time and an optional end time set when broken by a block (Assumption 7). Validity intervals allow historical follower counts at any past timestamp.
Block			✓	A block is a discrete event. The brief does not describe unblocking, so blocks are immutable records.
Mute		✓		A mute can be lifted, requiring start and optional end times. Same pattern as Follow.
Impression			✓	An impression is an immutable event. Feed position and model version are captured at impression time and never updated (Assumption 9).

Entity	O	V	A	Justification
ViewSegment			✓	Once a playback segment closes (user scrolls away or clip ends), it is immutable. A sequence of segments under one impression is naturally modelled as an append-only log.
Like			✓	A like is created once and retained even after retraction (Assumption 10). The like row is never modified.
LikeRetraction			✓	A retraction is a distinct negative event written once and never modified (Assumption 10).
ExplicitSignal			✓	Save, comment, follow-from-feed, not-interested, and report are timestamped events. None are described as retractable or editable in the brief.
Share			✓	A share records a single user action at a single timestamp to one destination. It is an immutable event.
AgentSession	✓			A session is a container; its metadata (user, started_at) is written once and does not change.
Turn	✓			Turn content (user message, assistant message, token counts, model, temperature) is written once. Quality scores and user ratings are separate linked entities.
PromptTemplate			✓	Every edit to a template produces a new version row with its exact text. Past rows are never modified, ensuring every historical turn remains linked to the exact template text in use at the time (brief §2.5).
ModelPricing		✓		Token rates change when vendors reprice. Validity intervals allow the exact rate in effect at any past turn timestamp to be resolved by a bounded query on `valid_from`/`valid_to`, ensuring historical costs are never altered by a later price change (Assumption 11).
ToolCall			✓	Arguments, result, latency, and error status are recorded at execution time and never modified. The parent-call reference (Assumption 12) is also immutable.
Recommendation			✓	A recommendation is written when the agent returns a shelf and never modified. The link to a later impression is recorded as a nullable FK on Impression, not by updating the Recommendation row.
TurnQualityScore			✓	An LLM judge evaluation is a one-time event. If a turn is re-evaluated, a new score row is appended rather than overwriting, preserving evaluation history.
UserTurnRating			✓	A thumbs-up or thumbs-down is an event. The brief does not state that ratings can be changed; each rating is retained as an immutable record.

B.3 — Three Hardest Design Decisions

- I considered storing both current moderation state in `Video` and full history in `ModerationDecision`. I rejected this because duplicated state can diverge; instead, the current state is derived from the latest append-only decision.
- I considered combining creator status and monetisation tier in one temporal entity. I rejected this because a user can become a creator before receiving a tier, so `CreatorPeriod` and

`CreatorTierPeriod` remain separate.

- I considered storing total watch time directly in `Impression`. I rejected this because one impression can contain multiple viewing segments; separate `ViewSegment` rows also preserve impressions that were shown but never watched.

Deliverable C – Logical Design and Relational Algebra

C.1 – Relation Listing

Key constraints shown; full DDL in `schema.sql`.

Relation	PK	Foreign keys	Notable domains / constraints
§2.1 – Users			
AppUser	user_id	–	auth_type CHECK IN ('phone','google'); handle UNIQUE COLLATE NOCASE; auth_value UNIQUE. auth_type+auth_value replace AuthIdentity (Assumption 1).
InterestCategory	category_id	–	name UNIQUE.
DeclaredInterest	(user_id, category_id)	user_id → AppUser CASCADE; category_id → InterestCategory CASCADE	Composite PK encodes weak-entity semantics.
InferredInterest	inference_id	user_id → AppUser CASCADE; category_id → InterestCategory CASCADE	confidence REAL CHECK BETWEEN 0.0 AND 1.0; suppressed INT CHECK IN (0,1).
AccountStatusPeriod	status_period_id	user_id → AppUser CASCADE	status CHECK IN ('active','deactivated','pending_deletion'); valid_to NULL = currently in this status.
§2.2 – Creators and Content			
CreatorPeriod	creator_period_id	user_id → AppUser CASCADE	valid_to NULL = currently a creator.
CreatorTierPeriod	tier_period_id	creator_period_id → CreatorPeriod CASCADE	tier FK → MonetisationTier (lookup table, extensible); valid_to NULL = current tier.
AudioTrack	track_id	original_video_id → Video SET NULL DEFERRABLE	track_type CHECK IN ('original','licensed'); original_video_id NULL for licensed tracks.

Relation	PK	Foreign keys	Notable domains / constraints
Video	video_id	owner_id → AppUser RESTRICT; audio_track_id → AudioTrack SET NULL DEFERRABLE	duration_ms CHECK BETWEEN 20000 AND 90000; no current_state column (derived from ModerationDecision).
Hashtag	hashtag_id	–	tag UNIQUE (lower-cased at ingestion).
VideoHashtag	(video_id, hashtag_id)	video_id → Video CASCADE; hashtag_id → Hashtag CASCADE	–
ModerationDecision	decision_id	video_id → Video CASCADE	new_state FK → ModerationState (lookup table, extensible); actor_type CHECK IN ('classifier','human'); actor_id NULL for classifiers.
§2.3 – Social Graph			
Follow	follow_id	follower_id → AppUser CASCADE; followee_id → AppUser CASCADE	ended_at NULL = active; set by block, not deleted (Assumption 7).
Block	block_id	blocker_id → AppUser CASCADE; blocked_id → AppUser CASCADE	blocked_at NOT NULL.
Mute	mute_id	muter_id → AppUser CASCADE; muted_id → AppUser CASCADE	ended_at NULL = mute active.
§2.4 – Watch Telemetry			
Impression	impression_id	user_id → AppUser CASCADE; video_id → Video CASCADE; recommendation_id → Recommendation SET NULL DEFERRABLE	feed_position CHECK ≥ 1 (fixed at impression time, Assumption 9); recommendation_id NULL = organic ranking.
ViewSegment	(impression_id, segment_start_ms)	impression_id → Impression CASCADE	segment_end_ms CHECK > segment_start_ms; completed CHECK IN (0,1). Weak entity; watch time = SUM(end−start).
Like	like_id	user_id → AppUser CASCADE; video_id → Video CASCADE	liked_at NOT NULL.
LikeRetraction	retraction_id	like_id → Like CASCADE	like_id UNIQUE (at most one retraction per like).

Relation	PK	Foreign keys	Notable domains / constraints
ExplicitSignal	signal_id	user_id → AppUser CASCADE; video_id → Video CASCADE	signal_type CHECK IN ('save','comment','follow_from_feed','not_int
Share	share_id	user_id → AppUser CASCADE; video_id → Video CASCADE	destination CHECK IN ('whatsapp','instagram','copied_link').
§2.5 – Agent Layer			
AgentSession	session_id	user_id → AppUser CASCADE	–
PromptTemplate	template_id	–	UNIQUE(template_name, version); append-only, past rows never modified.
ModelPricing	pricing_id	–	UNIQUE(model_name, vendor, valid_from); all rates CHECK ≥ 0; valid_to NULL = currently in effect.
Turn	turn_id	session_id → AgentSession CASCADE; template_id → PromptTemplate RESTRICT; pricing_id → ModelPricing RESTRICT	UNIQUE(session_id, turn_order); turn_order CHECK ≥ 1; pricing_id frozen at insert time (Assumption 11); RESTRICT prevents deletion of templates or pricing rows that have turns.
ToolCall	call_id	turn_id → Turn CASCADE; parent_call_id → ToolCall SET NULL DEFERRABLE	arguments TEXT CHECK json_valid (E.4 JSON use); errored CHECK IN (0,1); parent_call_id NULL = root call (Assumption 12).
Recommendation	recommendation_id	turn_id → Turn CASCADE; video_id → Video RESTRICT	shelf_position CHECK ≥ 1; RESTRICT prevents silent deletion of audit trail.
TurnQualityScore	score_id	turn_id → Turn CASCADE	helpfulness, groundedness, safety each CHECK BETWEEN 0.0 AND 5.0 (nullable).
UserTurnRating	rating_id	turn_id → Turn CASCADE	rating CHECK IN ('up','down'); append-only.

C.2 – Mapping Decision Log

ER construct	Mapping applied	Alternative rejected	Why
AppUser ↔ AuthIdentity (1:1)	Two attributes folded into AppUser: `auth_type`, `auth_value`	Separate AuthIdentity table with FK to AppUser	Assumption 1 guarantees exactly one identity per user, so there is no cardinality reason to create a separate table. A separate table would add a join to every authentication lookup with no modeling benefit.
Video ↔ Hashtag (M:N)	Junction table VideoHashtag with composite PK (video_id, hashtag_id)	Array column on Video storing hashtag strings	The brief requires hashtag-based queries (F5: find all videos with a given hashtag). An array column prevents efficient lookup and makes normalization of case inconsistencies impossible without full-table scans.
AppUser ↔ InterestCategory declared (M:N)	Junction table DeclaredInterest with composite PK (user_id, category_id)	Comma-separated list in AppUser	Relational design requires each interest to be independently addressable. A list column prevents FK enforcement and makes per-category queries impossible.
InferredInterest (temporal, M:N with history)	Separate relation with surrogate PK inference_id; validity carried by week_start	Overwrite the DeclaredInterest row with latest inference	Append-only strategy required by Assumption 3: the ML team must reconstruct the exact confidence at any past week. Overwriting destroys that information.
AppUser ↔ AppUser follow (M:N, temporal)	Follow table with surrogate PK follow_id, follower_id, followee_id, started_at, ended_at	Delete the row when a block occurs	Assumption 7: a block sets ended_at rather than deleting, preserving historical follower-count queries. A hard delete loses the fact that the follow existed.
AppUser ↔ AppUser block (M:N event)	Block table with surrogate PK block_id	Flag column blocked on AppUser	A flag cannot record multiple block events between the same pair of users over time, cannot record the timestamp of each, and cannot support queries over block history.
ViewSegment (weak entity)	Composite PK (impression_id, segment_start_ms); impression_id is both PK and FK	Surrogate PK segment_id with impression_id as FK only	ViewSegment is a weak entity: it has no meaning outside its parent Impression. Composite PK makes the dependency explicit and prevents a segment from being accidentally re-pointed to a different impression. A surrogate key was rejected because it hides the weak-entity semantics.
ModerationDecision (append-only event log)	Separate event-log table; current state derived from `MAX(decided_at)`	current_state column on Video updated in place, with history in a separate log	The dual-write failure in §2.7 arises precisely from updating Video.current_state and inserting a history row in two separate writes. Removing the current_state column from Video eliminates the anomaly entirely: there is only one write.

ER construct	Mapping applied	Alternative rejected	Why
Like + LikeRetraction (two separate event tables)	Like records the positive signal; LikeRetraction records the negative signal independently; FK from LikeRetraction to Like	Single Like table with a retracted flag or a retracted_at column	The brief explicitly states a retraction is "a negative signal in its own right, not merely the absence of a like" (§2.4). A flag conflates two distinct events into one row, preventing the G.3 transaction demonstration from writing each signal as its own atomic event.
CreatorPeriod + CreatorTierPeriod (two temporal tables)	CreatorPeriod records the creator status interval; CreatorTierPeriod records the tier interval, FK to CreatorPeriod	Single CreatorTierPeriod with user_id, collapsing creator status into tier existence	A user may become a creator before being assigned any monetisation tier. If CreatorPeriod is collapsed, the period between creator activation and first tier assignment is unrepresentable. The two tables are kept separate to allow a creator window with no tier.
PromptTemplate (append-only)	Every edit creates a new row with a new template_id; past rows never modified	Versioned table with mutable current text and a separate history table	The brief requires every historical turn to remain linked to the exact template text in use at the time (§2.5). If the current text is mutable, a careless update could alter the meaning of past turns. Append-only makes past rows immutable by design.
ModelPricing (validity intervals)	pricing_id FK on Turn frozen at insert time; Turn.pricing_id points to the rate in effect when the turn occurred	Compute cost at query time by joining to the current price row	Assumption 11: Finance requires that a vendor price change must never alter a past quarter's reported cost. Computing at query time against a mutable price table violates this requirement. Freezing the FK at insert time is the schema consequence the brief asks the reader to find.
ToolCall self-reference (nesting)	parent_call_id as nullable FK on ToolCall pointing to ToolCall(call_id)	Separate ToolCallNesting junction table (call_id, parent_call_id)	A tool call has at most one parent (Assumption 12), so a junction table would add a row for a relationship that is already a simple FK. The self-reference supports arbitrary-depth recursive CTE traversal (F9) with a single parent pointer.
Recommendation → Impression (critical link)	Nullable FK recommendation_id on Impression pointing to Recommendation	FK on Recommendation pointing to Impression	The Recommendation is written when the agent returns its shelf. The Impression is written later, when the user is actually shown the clip. The FK must be on the later entity (Impression) because the earlier entity (Recommendation) cannot know its future impression ID at insert time.

ER construct	Mapping applied	Alternative rejected	Why
JSON for ToolCall.arguments	TEXT column with CHECK(json_valid(arguments))	Structured columns per tool (query, search_query, filter_type…)	The brief explicitly lists several different tools (search_videos, get_user_history, fetch_trending_audio), each with different argument shapes. The argument structure genuinely varies per tool and cannot be normalized into a fixed set of columns without either a very wide sparse table or a further entity per tool type. This is the one defensible JSON use per E.4. What is lost: no FK into the JSON, no CHECK on interior fields.

C.3 – Relational Algebra

Notation: σ selection, π projection, $\bowtie$ natural join, $\bowtie$ left outer join, $\mathcal{G}$ groupby/aggregate, ρ rename.

F2 – Watch hours and completion rate per creator, live clips only; every creator must appear

Intent. Every user who has ever held a CreatorPeriod must appear in the result, even if they have no live videos, no impressions, or no views. The grain is one row per creator.

Step 1 – Determine the current moderation state of each video.

Since `ModerationDecision` is append-only, a video may have multiple historical decisions. Therefore, selecting every row with `new_state = 'live'` would be incorrect because a video that was live in the past may no longer be live.

Define the latest decision time for each video:

$$\text{LatestDecision} \leftarrow \mathcal{G}_{\textit{video_id};\ \max(\text{decided_at})\rightarrow\text{latest_time}}(\text{ModerationDecision})$$

Then retain only the decision that occurred at that latest time:

$$\text{CurrentModeration} \leftarrow \text{LatestDecision} \bowtie \text{ModerationDecision}$$

$$\text{LiveVideo} \leftarrow \pi_{\text{video_id}}\left(\sigma_{\text{new_state='live'}}(\text{CurrentModeration})\right)$$

Step 2 – Compute watch time and completion per impression.

A single impression may contain multiple viewing segments. Therefore, completion must be calculated at the impression level using max(completed), rather than averaging the segment-level values.

$$\text{ImpressionStats} \leftarrow \mathcal{G}_{\textit{impression_id};\ \sum(\text{segment_end_ms}-\text{segment_start_ms})\rightarrow\text{watch_ms},\ \max(\text{completed})\rightarrow\text{completed}}(\text{ViewSeg}$$

Step 3 – Join live videos to their impressions and views.

$$\text{WatchData} \leftarrow \text{LiveVideo} \bowtie \text{Video} \bowtie \text{Impression} \bowtie \text{ImpressionStats}$$

Step 4 – Preserve creators with no matching watch data.

$$\text{CreatorBase} \leftarrow \rho_{c_id \leftarrow \text{user_id}}(\text{CreatorPeriod})$$

$$\text{Result} \leftarrow \pi_{c_id,\ \text{watch_hrs},\ \text{completion_rate}} \left(\mathcal{G}_{c_id;\ \text{SUM(watch_ms)/3,600,000} \rightarrow \text{watch_hrs},\ \text{AVG(completed)} \rightarrow \text{completion_rate}} (\text{C}\right.$$

For creators with no matching impressions, the outer join preserves the creator row; the aggregate can be interpreted as zero watch hours and zero completion rate.

Why the rewrite is correct and cheaper. The moderation history must first be reduced to the latest decision per video. Otherwise, a video that was live at any earlier time would be incorrectly included.

The selection for the live state is therefore applied after finding the latest moderation decision but before joining with the large `Impression` and `ViewSegment` relations. This reduces the amount of data entering the expensive watch-data join.

F3 – Videos with no audio track (NOT IN vs NOT EXISTS)

NOT IN form:

$$\pi_{\text{video_id}} \left(\sigma_{\text{video_id} \notin \pi_{\text{video_id}}\left(\sigma_{\text{audio_track_id IS NOT NULL}}(\text{Video})\right)}(\text{Video})\right)$$

NOT EXISTS form:

In tuple-calculus form, the equivalent condition is:

$$\pi_{\text{video_id}} \left(\sigma_{\neg \exists t \in \text{AudioTrack}\ [\text{Video.audio_track_id} = t.\text{track_id}]}(\text{Video})\right)$$

Equivalently, because the schema stores the audio relationship directly as a nullable foreign key, the same result can be expressed more simply in relational algebra as:

$$\pi_{\text{video_id}} \left(\sigma_{\text{audio_track_id IS NULL}}(\text{Video})\right)$$

Note on NULL behaviour. The NOT IN form is sensitive to SQL's three-valued logic. If the subquery can return a NULL value, the comparison can become UNKNOWN rather than TRUE, potentially causing rows to be excluded unexpectedly. Here, the subquery explicitly filters to `audio_track_id IS NOT NULL`, so no NULL is returned by the subquery and the NOT IN form is safe.

The NOT EXISTS form does not have this problem because it checks whether a matching row exists rather than comparing against a possibly-NULL set of values.

For this particular schema, the direct `audio_track_id IS NULL` selection is the simplest relational algebra expression and produces the same result.

F12 – Sessions where the agent recommended a clip that the user then watched to completion

Intent. The result includes, for each qualifying session and turn, the recommended video and its shelf position.

The critical link is the chain:

$$\text{Turn} \rightarrow \text{Recommendation} \rightarrow \text{Impression} \rightarrow \text{ViewSegment}.$$

Step 1 – Find impressions whose video was watched to completion.

Because completion for an impression is defined as reaching the end in at least one of its viewing segments, it is sufficient to find impressions having at least one segment with `completed = 1`:

$$\text{CompletedImpression} \leftarrow \pi_{\text{impression_id}}\left(\sigma_{\text{completed}=1}(\text{ViewSegment})\right)$$

Step 2 – Link completed impressions back to recommendations.

$$\text{WatchedRecommendation} \leftarrow \pi_{\text{recommendation_id}}\left(\text{CompletedImpression} \bowtie \sigma_{\text{recommendation_id IS NOT NULL}}(\text{Impre}\right.$$

This retains only recommendation IDs for which at least one corresponding impression contains a completed viewing segment.

Step 3 – Recover the session, turn, video, and shelf position.

$$\text{Result} \leftarrow \pi_{\text{session_id, turn_id, video_id, shelf_position}}\left(\text{AgentSession} \bowtie \text{Turn} \bowtie (\text{Recommendation} \bowtie \text{WatchedRec}\right.$$

Why this is correct. Only recommendations with a completed impression appear, because the inner join on `WatchedRecommendation` excludes those never shown or never completed; the nullable `Impression.recommendation_id` FK is the load-bearing structural element.

Deliverable D – Normalisation

D.1 – Normal Form Analysis

Four relations are analysed in full. At least one is from the agent layer (**ModelPricing**, **Turn**) and at least one is temporal (**ModelPricing**, **CreatorTierPeriod**, **InferredInterest**).

The three-bucket method is used: Bucket L (LHS-only attributes, must appear in every key), Bucket R (RHS-only, in no key), Bucket B (both/neither).

Relation 1: ModelPricing

Schema.

ModelPricing(pricing_id, model_name, vendor, input_rate, output_rate, cached_input_rate, valid_from,

Functional dependencies claimed.

- **F1:** pricing_id → model_name, vendor, input_rate, output_rate, cached_input_rate, valid_from, valid_t
  Holds because: `pricing_id` is a surrogate key assigned at insert time. One pricing record represents one vendor's rates for one model during one validity interval. All attributes are properties of that single record.
- **F2:** {model_name, vendor, valid_from} → pricing_id, input_rate, output_rate, cached_input_rate, valid
  Holds because: A vendor publishes one set of token rates for a given model starting at a given effective date. Therefore, two pricing rows for the same model, vendor, and `valid_from` timestamp would represent the same pricing interval and are not permitted. Hence {model_name, vendor, valid_from} uniquely identifies a pricing record.

Three-bucket classification.

- Bucket L (LHS only, never RHS): {`model_name`, `vendor`, `valid_from`} — these appear only as determinants.

- Bucket R (RHS only, never LHS): {`input_rate`, `output_rate`, `cached_input_rate`, `valid_to`} — these are determined attributes.
- Bucket B (both sides): {`pricing_id`} — appears on the LHS of F1 and the RHS of F2.

Candidate key discovery. Bucket L attributes must be in every candidate key. Compute {model_name, vendor, valid_from}$^+$:

{model_name, vendor, valid_from} $\xrightarrow{F2}$ {model_name, vendor, valid_from, pricing_id, input_rate, output_

This equals all attributes. Therefore {model_name, vendor, valid_from} is a superkey. No proper subset of it is a superkey, so {model_name, vendor, valid_from} is a **candidate key**.

Now check {pricing_id}$^+$ via F1: {pricing_id}$^+$ = all attributes. So `pricing_id` is also a **candidate key**.

Candidate keys: {`pricing_id`} and {`model_name`, `vendor`, `valid_from`}.

BCNF verdict. For a relation to be in BCNF, every non-trivial FD $X \to Y$ must have X as a superkey. Check each FD:

- F1: LHS = {`pricing_id`}, which is a candidate key. ✓
- F2: LHS = {`model_name`, `vendor`, `valid_from`}, which is a candidate key. ✓

No FD violates BCNF. **ModelPricing is in BCNF.**

Relation 2: InferredInterest

Schema.

InferredInterest(<u>inference_id</u>, user_id, category_id, confidence, week_start, suppressed)

Functional dependencies claimed.

- **F1:** inference_id → user_id, category_id, confidence, week_start, suppressed
  Holds because: `inference_id` is a surrogate key assigned at insert time to each weekly ML inference event. All attributes are properties of that single inference run.
- **F2:** {user_id, category_id, week_start} → inference_id, confidence, suppressed
  Holds because: The ML pipeline runs once per week per user per interest category and appends one row per run. Running twice in the same week for the same user-category pair would produce two conflicting confidence scores, which the ML team explicitly does not do. Therefore the triple {user_id, category_id, week_start} identifies at most one inference record.
- **F3:** Does user_id → suppressed? *No* — suppression is per user-category pair, not per user. A user may suppress one inferred interest while retaining others.
- **F4:** Does confidence → category_id? *No* — confidence is a real-valued score; many categories may share the same confidence value for different users and weeks.

Three-bucket classification.

- Bucket L (LHS only): {user_id, category_id, week_start}
- Bucket R (RHS only): {confidence, suppressed}
- Bucket B (both): {inference_id}

Candidate key discovery. Compute {user_id, category_id, week_start}$^+$:

{user_id, category_id, week_start} $\xrightarrow{F2}$ {user_id, category_id, week_start, inference_id, confidence, suppres

This equals all attributes. No proper subset is a superkey (dropping any one attribute leaves the set unable to determine inference_id via any FD). Therefore {user_id, category_id, week_start} is a **candidate key**.

{inference_id}$^+$ $\xrightarrow{F1}$ all attributes. `inference_id` is also a **candidate key**.
Candidate keys: {`inference_id`} and {`user_id`, `category_id`, `week_start`}.
BCNF verdict.

- F1: LHS = {inference_id} — candidate key. ✓
- F2: LHS = {user_id, category_id, week_start} — candidate key. ✓

InferredInterest is in BCNF.

Relation 3: CreatorTierPeriod

Schema.

CreatorTierPeriod(tier_period_id, creator_period_id, tier, valid_from, valid_to)

Functional dependencies claimed.

- **F1:** tier_period_id → creator_period_id, tier, valid_from, valid_to
  Holds because: `tier_period_id` is a surrogate key. Each row is one continuous interval during which a creator occupied one tier, so all attributes are properties of that single record.
- **F2:** {creator_period_id, valid_from} → tier_period_id, tier, valid_to
  Holds because: A creator transitions to at most one monetisation tier at any given moment. Two tier-period records for the same creator period beginning at the same `valid_from` timestamp would mean the creator was assigned two different tiers simultaneously, which the business does not allow. Finance's requirement to answer "which tier at a past date?" is grounded in this uniqueness: the interval structure is only meaningful if intervals do not overlap.
- **F3:** Does tier → valid_to? *No* — the same tier value (e.g. `'rising'`) may end on many different dates for different creators.
- **F4:** Does creator_period_id → tier? *No* — a creator can hold different tiers over the course of one creator period (the tier changes as their audience grows).

Three-bucket classification.

- Bucket L (LHS only): {creator_period_id, valid_from}
- Bucket R (RHS only): {valid_to}
- Bucket B (both): {tier_period_id, tier}

Candidate key discovery. Start with Bucket L: {creator_period_id, valid_from}$^+$:

{creator_period_id, valid_from} $\xrightarrow{F2}$ {creator_period_id, valid_from, tier_period_id, tier, valid_to}

All attributes. No proper subset is a superkey. Therefore {creator_period_id, valid_from} is a **candidate key**.

$\{\text{tier_period_id}\}^+ \xrightarrow{F1}$ all attributes. **tier_period_id** is also a **candidate key**.

Check Bucket B remainder — {tier}: $\{\text{tier}\}^+ = \{\text{tier}\}$ only (no FD has tier alone as LHS). Not a superkey. Correct.

Candidate keys: {`tier_period_id`} and {`creator_period_id`, `valid_from`}.

BCNF verdict.

- F1: LHS = {tier_period_id} — candidate key. ✓
- F2: LHS = {creator_period_id, valid_from} — candidate key. ✓

CreatorTierPeriod is in BCNF.

Relation 4: Turn

Schema.

Turn(turn_id, session_id, template_id, pricing_id, turn_order, user_message, assistant_message, model_

Functional dependencies claimed.

- **F1:** turn_id → all other attributes
  Holds because: `turn_id` is a surrogate key. Each turn is one exchange within one session, generated once, with a fixed assistant response and a fixed token count. All attributes are properties of that single exchange.
- **F2:** {session_id, turn_order} → all other attributes
  Holds because: Within a single conversational session, turns are numbered sequentially and `turn_order` is unique per session. Two turns in the same session cannot share the same position. Therefore {session_id, turn_order} uniquely identifies a turn and determines all its attributes.
- **F3:** Does pricing_id → model_name or pricing_id → input_rate?
  No — `pricing_id` is a frozen FK pointing to a row in `ModelPricing`. The rates (input_rate, output_rate, cached_input_rate) live on ModelPricing, not on Turn. Turn stores only token *counts*. Therefore pricing_id determines nothing within the Turn relation itself beyond being an FK value. This is precisely why the design is correct: splitting rates into ModelPricing eliminates the transitive dependency pricing_id → rates from Turn entirely.
- **F4:** Does model_name → temperature?
  No — the same model (e.g. `gpt-4o-mini`) may be called at different temperatures in different turns. Temperature is a per-turn choice, not a property of the model.
- **F5:** Does template_id → model_name?
  No — the same prompt template may be used with different models over time (e.g. a template tested with `llama-3.1-70b` and later with an internal fine-tune). Model name is recorded per turn, not per template.

Three-bucket classification.

- Bucket L (LHS only): {`turn_id`, `session_id`, `turn_order`} — these attributes appear on the left-hand side of an FD and never on the right-hand side.
- Bucket R (RHS only): {`template_id`, `pricing_id`, `user_message`, `assistant_message`, `model_name`, `temperature`, `input_tokens`, `output_tokens`, `cached_tokens`, `responded_at`} — these are determined attributes.

- Bucket B (both sides): $\varnothing$.

Candidate key discovery. Bucket L attributes must appear in every candidate key. Compute {session_id, turn_order}$^+$:

$$\{\text{session_id, turn_order}\} \xrightarrow{F2} \text{all attributes}$$

No proper subset of {session_id, turn_order} is a superkey (session_id alone determines nothing within Turn; turn_order alone determines nothing). Therefore {session_id, turn_order} is a **candidate key**.

{turn_id}$^+ \xrightarrow{F1}$ all attributes. `turn_id` is also a **candidate key**.

Check Bucket B remainders individually:

- {template_id}$^+$ = {template_id} only. Not a superkey.
- {pricing_id}$^+$ = {pricing_id} only. Not a superkey.
- {model_name}$^+$ = {model_name} only. Not a superkey.

Candidate keys: {`turn_id`} and {`session_id`, `turn_order`}.

BCNF verdict.

- F1: LHS = {turn_id} — candidate key. ✓
- F2: LHS = {session_id, turn_order} — candidate key. ✓
- F3–F5: All shown to be *non-FDs* by domain reasoning above. No spurious FDs exist within Turn because rates, template text, and model properties live in their respective referenced relations, not in Turn itself.

Turn is in BCNF.

D.2 – The Denormalisation Not Done

The temptation: `current_state` on Video

The brief names this pattern explicitly (§2.7): the founders stored moderation history in one table and the current state in a column on Video, and roughly once a week one of the two writes landed while the other did not.

The query it would accelerate. Trust & Safety's view `v_video_current_state` currently derives the current moderation state by finding the `ModerationDecision` row with the maximum `decided_at` for each video:

```
SELECT video_id, new_state
FROM   ModerationDecision AS md
WHERE  decided_at = (
         SELECT MAX(decided_at)
         FROM   ModerationDecision
         WHERE  video_id = md.video_id);
```

This is a correlated subquery (or equivalent window function) over potentially millions of moderation events. Storing `current_state` directly on Video would reduce this to a single indexed column lookup with no subquery.

The update anomaly it would admit. A `current_state` column on Video must be updated every time a new `ModerationDecision` is inserted. This requires two writes in one logical operation:

1. `INSERT INTO ModerationDecision ...`

2. `UPDATE Video SET current_state = ?  WHERE video_id = ?`

If the process is killed, the connection drops, or a pod restarts between these two writes, the database enters an inconsistent state: `ModerationDecision` records the new state but `Video.current_state` still shows the old one — or vice versa. This is precisely the anomaly described in §2.7 of the brief. The inconsistency is not detectable from either table alone; only by comparing them does the contradiction become visible.

The enforcement mechanism required. A trigger or transaction could enforce the invariant, but it adds hidden database-side logic and every update path must preserve it.

Why it was not implemented. The adopted solution — deriving current state from `v_video_current_state` — eliminates the anomaly without duplicating state.

Deliverable E – Physical Implementation

E.1 Notes – Design Decisions in `schema.sql`

Pragmas

The following two pragmas appear at the top of `schema.sql` and are also set on the generator's database connection:

```
PRAGMA foreign_keys = ON;   -- per-connection; must be set on every
   connection
PRAGMA journal_mode = WAL;  -- required for G.3 concurrency
   demonstration
```

Type-Affinity Check

SQLite maps every declared type name onto one of five storage classes (NULL, INTEGER, REAL, TEXT, BLOB) by a substring rule, not by the exact name. The test in `schema.sql`:

```
CREATE TABLE _type_affinity_test (id INTEGER PRIMARY KEY, val INTEGER);
INSERT INTO _type_affinity_test VALUES (1, 'hello');
-- SELECT typeof(val) FROM _type_affinity_test WHERE id = 1;
-- Result: 'text'   <-- not an error; SQLite stored the string
```

Inserting the text value `'hello'` into a column declared `INTEGER` does not raise an error. SQLite stores it as text and `typeof(val)` returns `'text'`.

Consequence for this schema. Type declarations are hints to the query planner, not enforcement mechanisms. Every domain constraint that matters—clip duration, confidence bounds, valid state values—is enforced by a `CHECK` constraint, not by the type declaration.

Closed Value Sets: CHECK vs Lookup Table

Column	Strategy	Reason
`ModerationState.state`	Lookup table	Trust & Safety will add states (the brief implies this by noting state changes over a video's life). Adding a state requires only `INSERT INTO ModerationState`, not a schema migration. The set is also queryable: "how many videos are in each state?" becomes a simple join.
`MonetisationTier.tier`	Lookup table	Finance controls tier definitions and may add tiers as the product grows. Same extensibility argument as moderation state.
`auth_type`	`CHECK IN`	The brief specifies exactly two registration methods: phone and Google. This set is stable and small; a lookup table would add a join to every authentication query for no modeling benefit.
`signal_type`	`CHECK IN`	Five signal types, defined by the product specification. Extending this set would require a product change and a schema discussion in any case; a `CHECK` makes the constraint visible and cheap.
`destination` (Share)	`CHECK IN`	Three destinations (WhatsApp, Instagram, copied link) are named in the brief. Small, stable, product-defined.
`actor_type` (ModerationDecision)	`CHECK IN`	Two values only (classifier, human). Not expected to change.
`rating` (UserTurnRating)	`CHECK IN`	Binary (up/down). Cannot be extended without changing the product concept.

Handle Uniqueness

```
CREATE UNIQUE INDEX ux_handle_nocase
    ON AppUser(handle COLLATE NOCASE);
```

NOCASE and Tamil. SQLite's `NOCASE` collation handles ASCII case differences only; it does not provide general Unicode case-folding. Tamil itself has no upper/lower-case distinction, so `NOCASE` is not relevant for Tamil case differences. For scripts that do have Unicode case distinctions, application-level Unicode normalization/case-folding is needed.

Active-accounts-only uniqueness. A plain `UNIQUE` constraint cannot express "unique among active accounts". This rule is enforced in application code at two points: when a new handle is assigned at registration, and when a user changes their handle.

Timestamp Representation

All timestamps in this schema are stored as ISO-8601 text strings in the form `'2026-03-03T14:32:07Z'`.

Consequence for queries:

- **F9** (longest streak of consecutive active days): streak detection requires date arithmetic. ISO-8601 text must be converted via `julianday(ts)` to a numeric day number before consecutive-day comparisons can be made. `julianday` understands ISO-8601 format directly.

- **F10** (7-day rolling watch time): the `RANGE BETWEEN 6 PRECEDING AND CURRENT ROW` frame requires a numeric `ORDER BY` column. The query must use `ORDER BY julianday(impressed_at)` to produce numeric offsets that SQLite's range frame can compare.
- **Mixing representations** across tables is the single most common way this assignment fails at the query stage. Using ISO-8601 text uniformly prevents this.

NOT NULL Audit

Every nullable column is justified below. "Not yet known", "does not apply", and "user declined" are three distinct semantic states and should not share a representation; each nullable column belongs to exactly one of these categories.

Table	Column	What NULL means
AccountStatusPeriod	valid_to	Does not apply: the account is currently in this status; no end time exists yet.
CreatorPeriod	valid_to	Does not apply: the user is currently an active creator.
CreatorTierPeriod	valid_to	Does not apply: this is the creator's current tier.
AudioTrack	original_video_id	Does not apply: this is a licensed track with no originating video on the platform.
Video	audio_track_id	Does not apply: this video has no associated audio track (the brief states audio is optional).
Video	caption	User declined: the creator did not supply a caption.
ModerationDecision	actor_id	Does not apply: the decision was made by an automated classifier; no human reviewer ID exists.
Follow	ended_at	Does not apply: the follow relationship is currently active.
Mute	ended_at	Does not apply: the mute is currently active.
ModelPricing	valid_to	Does not apply: this pricing row is currently in effect.
Turn	temperature	Does not apply: the model was called with its default temperature; the caller did not specify a value.
ToolCall	parent_call_id	Does not apply: this is a root-level tool call issued directly by the assistant, with no parent call.
ToolCall	result	Does not apply: the call errored and returned no result; `errored = 1` in the same row.
ToolCall	latency_ms	Not yet known: latency measurement was unavailable for this call (e.g. the call timed out).
Impression	recommendation_id	Does not apply: this impression came from the organic ranking model, not from an agent recommendation.
TurnQualityScore	helpfulness, groundedness, safety	Does not apply: not every dimension is scored in every evaluation run; the judge may score a subset.

ON DELETE Behaviour

`ON DELETE` is chosen per relationship. Every non-default choice is commented in `schema.sql`. The rationale for the non-CASCADE choices is as follows:

- **Video.owner_id → AppUser: RESTRICT.** Deleting a user should not silently delete all their videos. The correct lifecycle is account deactivation, which is recorded in `AccountStatusPeriod` and leaves the video rows intact for moderation and audit purposes.
- **Video.audio_track_id → AudioTrack: SET NULL.** An audio track may be shared by up to 50,000 videos. Deleting one track should not cascade-delete all videos that use it; it should leave those videos without a track reference (NULL).
- **AudioTrack.original_video_id → Video: SET NULL.** The track can outlive its originating video. Other videos that reference the track continue to do so; only the originating-video pointer is cleared.
- **Recommendation.video_id → Video: RESTRICT.** A recommendation is an agent action recorded for audit. Silently deleting agent recommendations when a video is taken down would destroy the audit trail that Trust & Safety and the ML team need.
- **Turn.template_id → PromptTemplate: RESTRICT.** The brief requires every historical turn to remain linked to the exact template text used at the time. Deleting a template that has turns violates this requirement.
- **Turn.pricing_id → ModelPricing: RESTRICT.** Finance requires historical costs to be immutable. Deleting a pricing row that has turns would leave those turns without a cost basis.
- **ToolCall.parent_call_id → ToolCall: SET NULL.** If a parent call row were deleted (unusual but possible during data cleanup), child calls should survive with their parent reference cleared rather than being cascade-deleted. Cascade-deleting a parent would silently destroy the entire subtree, which is rarely the intended behaviour.

E.2 – Constraint Trace Table

Assumptions from A.1 that are enforceable, and where. Rules not expressible declaratively are explicitly noted.

#	Rule in words	Enforced by	Where
1	Exactly one registration identity per user; type is phone or Google	`CHECK (auth_type IN ('phone','google'))` + `NOT NULL` on `auth_value`	`AppUser`
2	Handle uniqueness is case-insensitive	`UNIQUE INDEX ...COLLATE NOCASE`	`ux_handle_nocase` on `AppUser`
2	Case-insensitive uniqueness covers ASCII only; Tamil handles not covered by NOCASE	Application-layer normalisation before insert	Not declarative
2	Uniqueness applies to active accounts only	Application code checks for conflicting active handle before assignment	Not declarative

#	Rule in words	Enforced by	Where
2	Handle may be changed at most twice per year	Application code counts handle-change log entries per user per calendar year	Not declarative
3	Handle remains reserved during deactivation and pending-deletion periods	Application code; account status is checked before releasing a handle	Not declarative
4	Creator-tier record represents a continuous validity period; no overwriting of past tiers	Append-only insert policy; no `UPDATE` path on `CreatorTierPeriod`	Application code; not declarative
5	Hashtags extracted and stored separately from caption	Separate `Hashtag` and `VideoHashtag` tables; caption stored as original text	`Hashtag`, `VideoHashtag`, `Video`
6	Moderation decisions retained as events; current state derived from latest decision	No `current_state` column on `Video`; state always derived from `ModerationDecision`	`ModerationDecision`; enforced structurally
7	Block sets `ended_at` on Follow rather than deleting the row	Application code; no `ON DELETE CASCADE` path from Block to Follow	Not declarative
8	Every impression recorded independently, even without a view	`Impression` and `ViewSegment` are separate tables; `ViewSegment` is optional per impression	`Impression`, `ViewSegment`
9	Feed position fixed at impression time	`feed_position NOT NULL` stored on `Impression`; no update path	`Impression`; application code
10	Original like retained; retraction stored as separate row	Separate `Like` and `LikeRetraction` tables; `UNIQUE` on `LikeRetraction.like_id` enforces at most one retraction per like	`LikeRetraction`
11	Token rates frozen at turn time (historical cost immutable)	`pricing_id` FK on `Turn` set at insert time; `ON DELETE RESTRICT` prevents rate deletion	`Turn.pricing_id`; `ModelPricing`
12	Tool call belongs to exactly one turn; nesting via `parent_call_id` self-reference	`turn_id NOT NULL` FK on `ToolCall`; `parent_call_id` nullable self-referencing FK	`ToolCall`

E.4 – JSON Column Justification

The schema uses JSON text only for `ToolCall.arguments`, guarded by `CHECK(json_valid(arguments))`. The `ToolCall.result` column remains ordinary `TEXT`.

Why JSON is defensible here. The brief lists several distinct tools: `search_videos`, `get_user_history`, `fetch_trending_audio`, and others. Each tool has a different argument signature. `search_videos` takes a query string and filters; `fetch_trending_audio` takes a region; `get_user_history` takes a lookback period. These cannot be mapped to a fixed set of relational columns without one of two bad outcomes:

- A very wide, sparse table with one column per possible argument across all tools, where most columns are NULL for any given row.
- A further entity (e.g. `ToolCallArgument(call_id, key, value)`) that loses type information and makes argument retrieval a pivot query.

The argument shape genuinely varies per tool and is not known in advance. This is the condition the brief describes as the "defensible candidate" for JSON (§E.4).

What is lost by choosing JSON.

1. **No foreign keys into the JSON.** If `arguments` contains a `video_id`, that value cannot reference `Video(video_id)` with a FK constraint. A tool call that references a deleted video will not be caught by the database.
2. **No `CHECK` on interior fields.** `json_valid` only confirms that the text is well-formed JSON; it cannot enforce that `arguments.limit` is a positive integer or that `arguments.region` is a valid ISO 3166-1 code. Interior validation must be performed by the application.

Deliverable F – Queries

All queries run on the database produced by `schema.sql` and `generate_data.py` (seed = "DA24B003", default scale). `SELECT sqlite_version()` returns `3.45.1`, confirming that `group_concat` with `ORDER BY` (F8), `RANGE` frames (F10), and recursive CTEs (F11) are all available.

Tier 1 — Core SQL

F1 · Top 10 audio tracks by distinct videos in the last 7 days

Intent. Rank audio tracks by how many unique videos using them received at least one impression in the most recent 7-day window. Tracks with no recent impressions are excluded; this is correct because the question asks about current trending reach.

Grain. One row per audio track. `COUNT(DISTINCT video_id)` prevents double-counting a video that received several impressions in the window.

Rows returned: 10 **Runtime:** 0.115s

First five rows:

```
track_id | distinct_videos
---------|----------------
    1431 |               9
    1400 |               9
    1764 |               8
    1508 |               8
    900 |                8
```

Reading. Top tracks reach 8–9 distinct videos; the distribution is flat, consistent with a small pool of equally popular audio.

F2 · Watch hours and completion rate per creator, live clips only; every creator must appear

Intent. Every user who has ever held a `CreatorPeriod` row must appear, even with zero watch hours. The brief states creators with zero activity must not be silently dropped.

Grain.

One row per creator.

Live-video detection. `ModerationDecision` is append-only; the current state is the `new_state` of the row with `MAX(decided_at)` per video.

Completion rate scope. `AVG(ss.completed)` is computed over *viewed* impressions only — impressions that produced at least one `ViewSegment` row. Impressions where the clip entered the viewport but was never held for 300 ms do not appear in `SegmentStats` and therefore do not reduce the average. This matches the brief's definition of a "view" as distinct from a bare "impression".

Rows returned: 714 (= number of creators) **Runtime:** 0.21s

First five rows:

```
creator_id | watch_hours | completion_rate
-----------|-------------|----------------
1912       | 1.4555      | 0.0092
3894       | 1.3945      | 0.0112
455        | 1.3229      | 0.0048
1261       | 1.3156      | 0.0047
746        | 1.2964      | 0.0120
```

Reading. 714 rows returned (= number of creators); the `LEFT JOIN` correctly preserves creators with no live-clip views at zero watch hours, confirming no silent drops.

F3 · Videos with no audio track — NOT IN vs NOT EXISTS

Intent. Find all videos where `audio_track_id` is `NULL`. Written in two forms to demonstrate the difference in NULL-safety.

Version A — NOT IN. Safe here because the subquery explicitly filters to `audio_track_id IS NOT NULL`, so the set presented to `NOT IN` contains no `NULL`s.

```
SELECT  video_id FROM Video
WHERE   video_id NOT IN (
    SELECT video_id FROM Video
    WHERE  audio_track_id IS NOT NULL
)
ORDER   BY video_id;
-- Rows returned: 6171   Runtime: 0.005s
```

Version B — NOT EXISTS. Immune to the `NULL` problem: it checks for the absence of a matching row rather than membership in a potentially `NULL`-containing set.

```
SELECT  v.video_id FROM Video AS v
WHERE   NOT EXISTS (
    SELECT 1 FROM AudioTrack AS at
    WHERE  at.track_id = v.audio_track_id
)
ORDER   BY v.video_id;
-- Rows returned: 6171   Runtime: 0.004 s
```

First five rows (both variants):

```
video_id
--------
       7
       9
      10
      12
      13
```

Row counts. Both return **6171**. They agree here because the `NOT IN` subquery returns only non-`NULL` `video_id` values; `video_id` is a primary key and therefore cannot be `NULL`. The nullable `audio_track_id` indicates that a video may have no audio track, but those `NULL` values are filtered out before the `NOT IN` comparison.

When they would diverge. If the subquery were `SELECT audio_track_id FROM Video` (without the `IS NOT NULL` filter), the result set would contain `NULL`s. SQL's three-valued logic then makes `video_id NOT IN (..., NULL, ...)` evaluate to `UNKNOWN` for every row, returning *zero rows* even though matches exist. `NOT EXISTS` is never affected because it does not compare against the returned values directly.

F4 · Users who liked and retracted within 60 seconds

Intent. Find (user, video, like) triples where the retraction followed the like within 60 seconds. Retraction is a separate event (`LikeRetraction`) joined to `Like` by `like_id`. Timestamps are ISO-8601 text; conversion to seconds uses `julianday()`.

`BETWEEN 0 AND 60` is used rather than `<= 60` to make the non-negative constraint explicit: a retraction cannot precede the like, so a negative diff signals a data anomaly and should be excluded, not silently included.

Rows returned: 616 **Runtime:** 0s

First five rows:

```
user_id | video_id | liked_at             | retracted_at         | secs
--------|----------|----------------------|----------------------|-----
292     | 17167    | 2025-01-06T10:24:07Z | 2025-01-06T10:24:07Z | 0
337     | 6774     | 2025-02-16T07:27:30Z | 2025-02-16T07:27:31Z | 0
4291    | 1448     | 2025-04-16T06:29:23Z | 2025-04-16T06:29:23Z | 0
1203    | 4815     | 2025-04-18T11:40:52Z | 2025-04-18T11:40:52Z | 0
1665    | 6055     | 2025-04-22T18:42:19Z | 2025-04-22T18:42:19Z | 0
```

Reading. All 616 results fall within 0–60 seconds; the explicit lower bound of 0 excludes any retraction timestamped before its like.

F5 · Videos whose caption carries a given hashtag (#cat), case-insensitively

Intent. Find videos whose caption contains the hashtag `#cat` as a whole token, regardless of case, with surrounding punctuation and whitespace tolerated. A caption containing the plain word "cat" without a leading `#` must *not* match. No `REGEXP` is available in SQLite; no `split_part` exists. All five required functions are used: `lower()`, `trim()`, `replace()`, `instr()`, and `LIKE`.

Strategy.

1. `lower()` — case-fold the caption so `#CAT` matches `#cat`.
2. `replace()` — replace non-hash punctuation (`,;.!?`) with spaces so hashtag tokens are cleanly delimited. The `#` is *not* replaced; stripping it would collapse `#cat` into `cat` and match plain words.
3. `trim()` — remove any leading or trailing whitespace left over after replacement.

4. `instr()` — quick pre-filter: `instr(norm_cap, '#cat') > 0` checks that the substring `#cat` appears at all before the more expensive boundary checks run.

5. `LIKE` — four branches enforce word-boundary delimiting: `'#cat %'` (at start), `'% #cat %'` (in middle), `'% #cat'` (at end), or `= '#cat'` (sole token).

Rows returned: 1501 **Runtime:** 0.025 s

Verified against `VideoHashtag`: both paths return exactly 1501. A caption containing the bare word "cat" without # does not match.

First five rows:

```
video_id | caption
---------|---------------------------------------
    9        Check this out! #dance #cat
    26        Check this out! #travel #satisfying #cat
    41        Check this out! #cat #memes #fashion
    86        Check this out! #trending #funny #cat
1   13        Check this out! #cat #fitness
```

LIKE vs GLOB; Tamil note. `LIKE` is used because SQLite's `LIKE` is case-insensitive for ASCII, so `lower() + LIKE '#cat %'` matches `#Cat`, `#CAT`, and so on without extra patterns. `GLOB` is case-sensitive and would require explicit bracket expressions such as `[#][Cc][Aa][Tt]`.

For a caption in Tamil script: Tamil has no upper/lower case distinction, so `lower()` is a no-op and ASCII LIKE case-folding does not apply. The `instr()` and `LIKE` checks still work correctly because the byte sequence of the Tamil hashtag token is identical regardless of any casing concern; the match is byte-for-byte, which is the correct behaviour for a script with no case.

F6 · Users shown a creator's clips but never engaged — set operator; UNION vs UNION ALL

Intent. Find users who received at least one impression of creator 461's clips but produced no like, explicit signal, or share on any of those clips. A set-difference operator (`EXCEPT`) expresses this directly.

Why UNION inside the EXCEPT operand. The engaged-user set is the union of three signal tables. Using `UNION` (not `UNION ALL`) deduplicates: a user who both liked and shared a clip appears only once in the engaged set, so `EXCEPT` correctly excludes them.

Rows returned: 27 **Runtime:** 0.12s

First five rows:

```
user_id
-------
473
544
754
1074
1097
```

UNION vs UNION ALL on the signal roll-up for creator 461:

```
-- UNION: deduplicated actors
SELECT user_id FROM Like  AS l JOIN Video AS v ON v.video_id=l.video_id
    WHERE v.owner_id=461
UNION
SELECT user_id FROM Share AS s JOIN Video AS v ON v.video_id=s.video_id
    WHERE v.owner_id=461;
```

```
-- Rows: 1

-- UNION ALL: all signal events, including repeat actors
SELECT user_id FROM Like  AS l JOIN Video AS v ON v.video_id=l.video_id
    WHERE v.owner_id=461
UNION ALL
SELECT user_id FROM Share AS s JOIN Video AS v ON v.video_id=s.video_id
    WHERE v.owner_id=461;
-- Rows: 1
```

Difference = 0.

F7 · Cost per agent session in the last 30 days, by template version, sessions above threshold

Intent. Compute the cost of each agent session in the last 30 days, broken down by the prompt template and version used in that session. Only sessions whose total cost exceeds 0.01 are returned.

Corrected cost formula. LLM vendors charge cached tokens at a reduced `cached_input_rate` rather than the full `input_rate`. Crucially, cached tokens are already counted inside `input_tokens`: they represent the portion of the input that was served from the prompt cache. The correct formula therefore subtracts cached tokens from the full input before applying `input_rate`, then prices the cached portion separately:

(input_tokens−cached_tokens)×input_rate+cached_tokens×cached_input_rate+output_tokens×outpu

The previous formula `input_tokens * input_rate + cached_tokens * cached_input_rate` double-charged cached tokens by applying both rates to them, overstating costs by approximately 11% on this dataset.

Key design consequence. The rates come from `ModelPricing` via `Turn.pricing_id`, frozen at turn-insertion time. Joining to the current price would silently restate historical costs after every vendor price change — exactly the failure the Finance requirement guards against.

Rows returned: 1396 **Runtime:** 0.62s

First five rows:

session_id	template_name	version	session_cost
185	search_shelf	4	18.446601
583	conversational_search	1	13.333351
508	explain_why	1	13.196326
926	search_shelf	3	13.137791
1563	explain_why	4	12.436505

Reading. 1,396 session–template combinations exceeded the threshold; session 185 (`search_shelf` v4) is the costliest at $18.45, computed against historically frozen rates.

F8 · Videos with more than two moderation state changes, with full chronological transition sequence

Intent. Find all videos that have received more than two moderation decisions, and show the ordered sequence of resulting states.

SQLite version requirement. `group_concat(...ORDER BY ...)` requires SQLite 3.44+. On earlier versions the concatenation order is undefined and results would appear plausible while being silently wrong. This database runs SQLite 3.53.4 (`SELECT sqlite_version()`).

Rows returned: 1636 **Runtime:** 0.02s

First five rows:

```
video_id | n_decisions | state_sequence
---------|-------------|---------------------------------------
6        | 3           | pending -> live -> demoted
13       | 3           | pending -> live -> taken_down
23       | 3           | pending -> live -> age_restricted
26       | 3           | pending -> live -> age_restricted
64       | 3           | pending -> live -> demoted
```

Reading. 1,636 videos have more than two decisions; all five shown follow `pending` → `live` → `{demoted|taken_down|age_restricted}`, confirming the event log preserves chronological order.

Tier 2 — Window Functions and Recursion

F9 · Each user's longest streak of consecutive active days

Intent. For each user, find the longest run of calendar days on which they had at least one impression.

Method: the consecutive-group trick.

1. Find each distinct `(user_id, calendar_day)` pair.
2. Compute `ROW_NUMBER()` partitioned by user, ordered by day.
3. Subtract the row-number (as a day offset) from the date. Consecutive days produce the same group key because both the date and the row number advance by one each day.
4. Count the streak length per group; take the `MAX` per user.

ISO-8601 text parses correctly in SQLite's `date()` and `julianday()` functions.

Rows returned: 20 (top 20 shown) **Runtime:** 0.755s

First five rows:

```
user_id | longest_streak_days
--------|--------------------
2389    | 18
1508    | 16
4366    | 15
154     | 13
4581    | 12
```

Reading. User 2389 leads with an 18-day streak; top-5 users range from 12–18 consecutive active days.

F10 · Rank creators by 7-day rolling watch time, with week-over-week change

"7 days" definition. Seven consecutive calendar days ending on (and including) each day the creator had watch activity. This is different from "the 7 days on which the creator happened to have activity" — the former counts any watch time falling within the prior 6 calendar days plus the current day; the latter would skip calendar days with no activity. Growth analysts asked for calendar days.

SQLite RANGE frame workaround. SQLite's `RANGE BETWEEN n PRECEDING AND CURRENT ROW` accepts only numeric offsets, not `INTERVAL '7 days'`. The fix: `ORDER BY julianday(date(impressed_at))` in the window, then use `RANGE BETWEEN 6.0 PRECEDING AND CURRENT ROW`. Since Julian day values differ by exactly 1.0 per calendar day, this covers exactly 7 calendar days.

Week-over-week via self-join, not `LAG(..., 7)`. `LAG(rolling_7d_ms, 7)` steps back 7 *activity rows* for each creator — not 7 calendar days. A creator with only 3 active days in the past fortnight would receive the rolling total from 3 weeks ago, not 1 week ago. The correct approach is a self-join on `Rolling` at `day_jd - 7.0`: this retrieves the rolling total for exactly 7 calendar days earlier, returning `NULL` cleanly when no activity row exists on that date (sparse creator, no spurious prior value).

Rows returned: 20 (top 20 by latest rolling 7-day total) **Runtime:** 1.003 s
First five rows:

```
creator_id | rolling_7d_ms | prev_week_ms | pct_change | rnk
-----------|---------------|--------------|------------|----
3992       | 367220        | NULL         | NULL       | 1
2925       | 339733        | NULL         | NULL       | 2
2744       | 305389        | NULL         | NULL       | 3
455        | 291802        | NULL         | NULL       | 4
3939       | 287270        | NULL         | NULL       | 5
```

Reading. Creator 3992 leads with ∼367 s rolling watch time; `NULL` in `prev_week_ms` correctly indicates no activity row exactly 7 calendar days prior (sparse creator history).

F11 · Full nesting tree for a given agent session's tool calls, with depth

Intent. Reconstruct the complete parent-child hierarchy of tool calls within one agent session. A recursive CTE traverses the self-referencing `parent_call_id` FK on `ToolCall`. The `path` column provides a dot-separated ancestry string that naturally sorts calls in tree order.

Rows returned: 9 (session 1) **Runtime:** 0.017s
All rows:

```
depth | call_id | parent | turn | tool_name        | err | latency | path
------|---------|--------|------|------------------|-----|---------|------
1     | 1       | NULL   | 1    | get_creator_info | 0   | 519     | 1
2     | 2       | 1      | 1    | search_videos    | 0   | 596     | 1.2
1     | 3       | NULL   | 1    | search_hashtags  | 0   | 30      | 3
1     | 4       | NULL   | 1    | search_hashtags  | 0   | 725     | 4
1     | 5       | NULL   | 2    | get_user_history | 0   | 135     | 5
```

Reading. Session 1 yields 9 calls; call 2 (path `1.2`) is the only nested child, confirming the recursive CTE correctly reconstructs depth from `parent_call_id`.

F12 · Sessions where the agent recommended a clip the user then watched to completion

Intent. Trace the critical link: `Turn` → `Recommendation` → `Impression` → `ViewSegment`. A recommendation "led to" a completion when the impression linked via `Impression.recommendation_id` contains at least one `ViewSegment` with `completed = 1`.

Schema note. If this query were hard to write, it would indicate a schema problem: the `Impression.recommendation_id` nullable FK is the structural feature that makes the causal chain expressible as a join. Without it, tracing "which recommendation led to which view" would require application-side correlation by user, video, and timestamp, which is fragile and slow.

Rows returned: 59 **Runtime:** 0.074s
First five rows:

```
session_id | turn_id | video_id | shelf_pos | completions
-----------|---------|----------|-----------|------------
18         | 80      | 9439     | 3         | 1
34         | 166     | 5087     | 1         | 1
52         | 255     | 13982    | 4         | 1
61         | 279     | 14730    | 1         | 1
64         | 293     | 14786    | 4         | 1
```

Reading. 59 recommendation-to-completion links found; completed views occur across multiple shelf positions, not only position 1.

F13 · Turns where the LLM judge scored above 4 but the user gave a thumbs-down

Intent. Find turns that the automated judge rated as high-quality (any dimension > 4.0) but the user explicitly rated negatively.

Rows returned: 43 **Runtime:** 0.032s

First five rows:

```
turn_id | session | helpfulness | groundedness | safety | rating | rated_at
--------|---------|-------------|--------------|--------|--------|----------
63      | 14      | 4.77        | 1.76         | 2.19   | down   | 2025-04-14T16:55:18Z
216     | 44      | 1.97        | 1.42         | 4.65   | down   | 2025-08-07T19:02:39Z
264     | 56      | 3.59        | 2.69         | 4.19   | down   | 2025-08-22T15:25:22Z
560     | 126     | 3.00        | 4.44         | 2.98   | down   | 2025-10-23T11:03:29Z
581     | 130     | 4.43        | 1.86         | 4.76   | down   | 2025-08-10T23:58:39Z
```

Reading. 43 turns had at least one LLM judge dimension above 4.0 but received a thumbs-down from the user. These disagreements highlight cases where the automated judge considered a response high-quality while the user did not, providing useful data for improving judge calibration, prompt templates, and the alignment between automated quality metrics and actual user satisfaction.

Why this disagreement set is the most commercially valuable data in the company. Every row is a labelled example where the automated judge was wrong about what the user valued — the highest-information-gain signal for correcting the feedback loop. The prompt-engineering team can use it to retrain the judge, revise the template that produced the turn, or detect systematic blind spots (e.g. technical correctness scored high while tone or personalisation failed) across a specific model or user segment.

Deliverable G – Views and Transactions

G.1 – Views (`views.sql`)

View Specification

View	Consumer	Columns exposed	Columns hidden	Depends on B.2 strategy	Updatable?
`v_public_profile`	Mobile client	`user_id`, `handle`, `display_name`, `follower_count`	`auth_value`, `auth_type`, `created_at`; accounts with non-active current status excluded	`AccountStatusPeriod` (V): current status derived from `valid_to IS NULL`	No (SQLite); Yes (PostgreSQL)

View	Consumer	Columns exposed	Columns hidden	Depends on B.2 strategy	Updatable?
`v_video_current_state`	Trust & Safety	`video_id`, `current_state`	All historical decisions, actor, timestamp	`ModerationDecision` (A): current state derived from `MAX(decided_at)`	No
`v_creator_tier_current`	Growth	`user_id`, `current_tier`	Validity intervals, creator_period_id	`CreatorTierPeriod` (V): current tier from `valid_to IS NULL`	No
`v_video_daily_engagement`	General analysts	`video_id`, `day`, `impression_count`, `view_count`, `watch_seconds`, `net_likes`	Raw segment offsets, like/retraction IDs	`ViewSegment` (A), `Like` (A), `LikeRetraction` (A)	No
`v_turn_cost`	Finance	`turn_id`, `session_id`, `model_name`, token counts, rates, `turn_cost`	Template text, raw pricing validity periods	`ModelPricing` (V): price frozen at turn insert via `pricing_id` FK	No

Design notes on each view

v_public_profile. Hides `AccountStatusPeriod` validity-interval strategy; filters `valid_to IS NULL AND status = 'active'` to exclude deactivated and deletion-window accounts; follower count excludes inactive followers.

v_video_current_state. Derives current state from `MAX(decided_at)` using the index on `(video_id, decided_at DESC)`; videos with no decision row return `'pending'` via `COALESCE`.

v_creator_tier_current. Current tier from `valid_to IS NULL`; creators with no tier row appear with `current_tier = NULL`.

v_video_daily_engagement. Impression is the spine; all signals `LEFT JOIN`ed to preserve days with impressions but zero engagement; `ViewSegment` aggregated per impression to prevent fan-out.

v_turn_cost. Simple equi-join on `Turn.pricing_id`, frozen at insert time; a vendor price change does not affect past rows.

G.2 – Written Answers

(a) Updatability of `v_public_profile`

Attempting an `INSERT` through `v_public_profile` in SQLite produces:

```
sqlite3.OperationalError: cannot insert into view 'v_public_profile'
```

SQLite views are always read-only. A view is *updatable in principle* when each view row maps unambiguously to one row of a single base table — requiring the primary key to be exposed and no aggregation, grouping, or set operators to be present. `v_public_profile` fails this because `follower_count` is an aggregate, so PostgreSQL would also reject an `INSERT` without an `INSTEAD`

OF trigger. v_video_daily_engagement is non-updatable in any engine: its GROUP BY means one view row aggregates many base rows and cannot be mapped back to a single one.

(b) The as-of problem

v_creator_tier_current returns tier as of *now* (valid_to IS NULL). The ML team needs tier as of any past timestamp T, which requires WHERE valid_from <= T AND (valid_to IS NULL OR valid_to >= T). A plain view cannot accept T as a parameter — the filter is hard-coded and a new view per timestamp is not a general solution. The correct mechanism is a parameterised query directly against CreatorTierPeriod with the above predicate, issued as a prepared statement by the ML pipeline.

G.3 – Transactions (transactions.sql and transactions.py)

T1 – Like retraction must be atomic

Failure mode. If the process dies after inserting the LikeRetraction row but before the transaction commits, an orphan retraction exists with no committed like state — or the retraction is lost entirely.

Demonstration. A duplicate INSERT INTO LikeRetraction for the same like_id is injected mid-transaction, violating the UNIQUE constraint and raising SQLITE_CONSTRAINT_UNIQUE. An explicit ROLLBACK undoes both inserts.

Consistency proof. After the failed transaction the Like row still exists (predates the transaction) and no LikeRetraction row exists. A subsequent successful transaction confirms both rows present after commit.

T2 – Moderation decision, WAL isolation

Failure mode eliminated. The founders required two writes (insert ModerationDecision and update Video.current_state); one could land without the other. This design has one write per moderation event; current state is always derived, so there is nothing to go out of sync.

WAL isolation (in transactions.py). Connection A inserts a new decision for video V but does not commit. Connection B queries v_video_current_state and sees the last committed snapshot — not A's pending write — without being blocked. After A commits, B queries again and sees the new state. In the default rollback-journal mode B would be blocked rather than shown the old value, which is why PRAGMA journal_mode = WAL is required.

T3 – Handle change and SQLITE_BUSY

Constraint demonstration. The twice-a-year limit is enforced in application code: before opening a transaction, the application queries v_handle_changes_this_year. The first two changes succeed; on the third attempt changes_this_year = 2 causes the application to reject the request without ever opening a transaction (E.2, Assumption 2).

SQLITE_BUSY demonstration (in transactions.py). While Connection A holds an open write transaction, Connection B attempts BEGIN IMMEDIATE and receives OperationalError: database is locked — the Python surface of SQLITE_BUSY. SQLite permits exactly one writer at a time, which sidesteps the **lost update** anomaly without requiring row-level locking or MVCC.